

- **Wd** - 숫자와 매치, [0-9]와 동일한 표현식이다.
- **WD** - 숫자가 아닌 것과 매치, [^{tab}^0-9]와 동일한 표현식이다.
- **Ws** - whitespace 문자와 매치, [^{return(커서이동)}WtWnWrWfWv]와 동일한 표현식이다. 맨 앞의 빈 칸은 공백문자(space)를 의미한다. WnWr은 엔터
- **WS** - whitespace 문자가 아닌 것과 매치, [^{newline}^WtWnWrWfWv]와 동일한 표현식이다.
- **Ww** - 문자+숫자(alphanumeric)와 매치, [a-zA-Z0-9]와 동일한 표현식이다.
- **WW** - 문자+숫자(alphanumeric)가 아닌 문자와 매치, [^a-zA-Z0-9]와 동일한 표현식이다.

대문자로 사용된 것은 소문자의 반대임을 추측할 수 있을 것이다.

Dot(.)

[a.b] 면 모두 통과됨.. 한 자리

정규 표현식의 Dot(.) 메타 문자는 줄바꿈 문자인 **Wn**를 제외한 모든 문자와 매치됨을 의미한다.

(※ 나중에 배우겠지만 정규식 작성 시 옵션으로 re.DOTALL이라는 옵션을 주면 **Wn** 문자와도 매치가 된다.)

다음의 정규식을 보자.

a.b a로 시작, b로 끝나는 3자리 문자

위 정규식의 의미는 다음과 같다.

"a + 모든문자 + b"

즉 a와 b라는 문자 사이에 어떤 문자가 들어가도 모두 매치된다는 의미이다.

이해를 돕기 위해 문자열 "aab", "a0b", "abc"가 정규식 a.b와 어떻게 매치되는지 살펴보자.

- "aab"는 가운데 문자 "a"가 모든 문자를 의미하는 .과 일치하므로 정규식과 매치된다.
- "a0b"는 가운데 문자 "0"가 모든 문자를 의미하는 .과 일치하므로 정규식과 매치된다.
- "abc"는 "a"문자와 "b"문자 사이에 어떤 문자라도 하나는있어야 하는 이 정규식과 일치하지 않으므로 매치되지 않는다.

※ 만약 앞에서 살펴본 문자 클래스([]) 내에 Dot(.) 메타 문자가 사용된다면 이것은 "모든 문자"라는 의미가 아닌 문자 . 그대로를 의미한다. 혼동하지 않도록 주의하자.

다음의 정규식을 보자.

a[.]b "a.b" 만 찾고 싶으면 [.]

이 정규식의 의미는 다음과 같다.

"a + Dot(.)문자 + b"

따라서 정규식 a[.]b 는 "a.b"라는 문자열과는 매치되고 "a0b"라는 문자와는 매치되지 않는다.

※ .는 **Wn**을 제외한 모든 문자와 매치되는데 심지어 **Wn**문자와도 매치되게 할 수도 있다. 나중에 알아보겠지만 정규식 작성시 옵션으로 **re.DOTALL** 이라는 옵션을 주면 **Wn**문자와도 매치되게 할 수 있다.

반복 (*)

다음의 정규식을 보자.

ca*t

이 정규식에는 반복을 의미하는 * 메타문자가 사용되었다. 여기서 사용된 *의 의미는 ***바로 앞에 있는 문자 a가 0부터 무한대로 반복될 수 있다는 의미이다.**

※ 여기서 * 메타문자의 반복갯수가 무한개까지라고 표현했는데 사실 메모리 제한으로 2억개 정도만 가능하다고 한다.

즉, 다음과 같은 문자열들이 모두 매치된다.

c, t는
있어야.

정규식	문자열	Match 여부	설명
ca*t	ct	Yes	"a"가 0번 반복되어 매치
ca*t	cat	Yes	"a"가 0번 이상 반복되어 매치 (1번 반복)
ca*t	caaat	Yes	"a"가 0번 이상 반복되어 매치 (3번 반복)

반복 (+)

반복을 나타내는 또 다른 메타 문자로 +가 있다. +는 **최소 1번 이상 반복될 때 사용한다.** 즉, *가 반복 횟수 0부터라면 +는 반복 횟수 1부터인 것이다.

다음 정규식을 보자.

ca+t

위 정규식의 의미는 다음과 같다.

"c + a(1번 이상 반복) + t"

위 정규식에 대한 매치여부는 다음 표와 같다.

정규식	문자열	Match 여부	설명
ca+t	ct	No	"a"가 0번 반복되어 매치되지 않음
ca+t	cat	Yes	"a"가 1번 이상 반복되어 매치 (1번 반복)
ca+t	caaat	Yes	"a"가 1번 이상 반복되어 매치 (3번 반복)

시작 끝  0개 or 1개
반복 ({m,n}, ?)

여기서 잠깐 생각해 볼 게 있다. 반복 횟수를 3회만 또는 1회부터 3회까지만으로 제한하고 싶을 수도 있지 않을까?

{ } 메타 문자를 이용하면 반복 횟수를 고정시킬 수 있다. {m, n} 정규식을 사용하면 반복 횟수가 m부터 n 까지인 것을 매치할 수 있다. 또한 m 또는 n을 생략할 수도 있다. 만약 {3,} 처럼 사용하면 반복 횟수가 3 이상인 경우이고 {,3} 처럼 사용하면 반복 횟수가 3 이하인 것을 의미한다. 생략된 m은 0과 동일하며, 생략된 n은 무한대(2억개 미만)의 의미를 갖는다.

※ {1,}은 +와 동일하며 {0,}은 *와 동일하다.

{ }을 이용한 몇가지 정규식을 살펴보자.

1. {m} m 번 반복

ca{2}t "caat" 만 통과

위 정규식의 의미는 다음과 같다.

"c + a(반드시 2번 반복) + t"

위 정규식에 대한 매치여부는 다음 표와 같다.

정규식	문자열	Match 여부	설명
ca{2}t	cat	No	"a"가 1번만 반복되어 매치되지 않음
ca{2}t	caat	Yes	"a"가 2번 반복되어 매치

2. {m, n}

ca{2,5}t

위 정규식의 의미는 다음과 같다:

"c + a(2~5회 반복) + t"

위 정규식에 대한 매치여부는 다음 표와 같다.

정규식	문자열	Match 여부	설명
ca{2,5}t	cat	No	"a"가 1번만 반복되어 매치되지 않음
ca{2,5}t	caat	Yes	"a"가 2번 반복되어 매치
ca{2,5}t	caaaaat	Yes	"a"가 5번 반복되어 매치

3. ? = {0,1} 0~1

반복은 아니지만 이와 비슷한 개념으로 ? 이 있다. ? 메타문자가 의미하는 것은 {0, 1} 이다.

다음의 정규식을 보자.

ab?c

위 정규식의 의미는 다음과 같다:

"a + b(있어도 되고 없어도 된다) + c"

위 정규식에 대한 매치여부는 다음 표와 같다.

정규식	문자열	Match 여부	설명
ab?c	abc	Yes	"b"가 1번 사용되어 매치
ab?c	ac	Yes	"b"가 0번 사용되어 매치

즉, b문자가 있거나 없거나 둘 다 매치되는 경우이다.

*****, **+**, **?** 메타문자는 모두 **{m, n}** 형태로 고쳐쓰는 것이 가능하지만 가급적 이해하기 쉽고 표현도 간결한 *****, **+**, **?** 메타문자를 사용하는 것이 좋다.

지금까지 아주 기초적인 정규표현식에 대해서 알아보았다. 정규식에 대해서 알아야 할 것들이 아직 많이 남아 있지만 그에 앞서 파이썬으로 이러한 정규표현식을 어떻게 사용할 수 있는지에 대해서 먼저 알아보기로 하자.

파이썬에서 정규 표현식을 지원하는 re 모듈

파이썬은 정규 표현식을 지원하기 위해 re(regular expression의 약어) 모듈을 제공한다. re 모듈은 파이썬이 설치될 때 자동으로 설치되는 기본 라이브러리로, 사용 방법은 다음과 같다.

```
>>> import re
>>> p = re.compile('ab*')
      패턴 문자열
      함수
      => 패턴 오브젝트 생성
```

re.compile 을 이용하여 정규표현식(위 예에서는 ab*)을 컴파일하고 컴파일된 패턴객체(re.compile의 결과로 리턴되는 객체 p)를 이용하여 그 이후의 작업을 수행할 것이다.

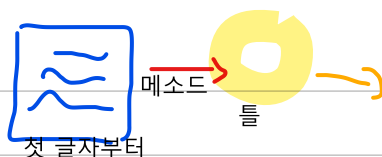
- ※ 정규식 컴파일 시 특정 옵션을 주는 것도 가능한데, 이에 대해서는 뒤에서 자세히 살펴본다.
- ※ 패턴이란 정규식을 컴파일한 결과이다.

정규식을 이용한 문자열 검색

이제 컴파일 된 패턴 객체를 이용하여 검색을 수행 해 보자. 컴파일 된 패턴 객체는 다음과 같은 4가지 메소드를 제공한다.

Method	목적
match()	문자열의 처음부터 정규식과 매치되는지 조사한다.
search()	문자열 전체를 검색하여 정규식과 매치되는지 조사한다.
findall()	정규식과 매치되는 모든 문자열(substring)을 리스트로 리턴한다

match()



Match object 나온다.
(list 등으로 후작업 필요)



`finditer()` 정규식과 매치되는 모든 문자열(substring)을 **iterator 객체로 리턴한다**

iterator object가 나옴
-> match object -> list

`match`, `search`는 정규식과 매치될 때에는 `match` 객체를 리턴하고 매치되지 않을 경우에는 `None`을 리턴한다. 이 메서드들에 대한 간단한 예를 살펴보자.

※ `match` 객체란 정규식의 검색 결과로 리턴되는 객체이다.

우선 다음과 같은 패턴을 만들어 보자.

```
>>> import re
>>> p = re.compile('[a-z]+')
```

match

`match` 메서드는 문자열의 처음부터 정규식과 매치되는지 조사한다. 위 패턴에 `match` 메서드를 수행해 보자.

```
>>> m = p.match("python")
>>> print(m)
<_sre.SRE_Match object at 0x01F3F9F8>
```

"python"이라는 문자열은 `[a-z]+` 정규식에 부합되므로 `match` 객체가 리턴된다.

```
>>> m = p.match("3 python")
>>> print(m)
None
```

"3 python"이라는 문자열은 처음에 나오는 3이라는 문자가 정규식 `[a-z]+`에 부합되지 않으므로 `None`이 리턴된다.

`match`의 결과로 `match` 객체 또는 `None`이 리턴되기 때문에 파이썬 정규식 프로그램은 보통 다음과 같은 흐름으로 작성한다.

```
p = re.compile(정규표현식)
m = p.match('string goes here')
if m:
    print('Match found: ', m.group())
else:
    print('No match')
```

즉, `match`의 결과값이 있을 때만 그 다음 작업을 수행하겠다는 의도이다.

search

컴파일된 패턴 객체 `p`를 가지고 이번에는 `search` 메서드를 수행해 보자.

```
>>> m = p.search("python")
>>> print(m)
<_sre.SRE_Match object at 0x01F3FA68>
```

"python"이라는 문자열에 `search` 메서드를 수행하면 `match` 메서드를 수행했을 때와 동일하게 매치된다.

```
>>> m = p.search("3 python")
>>> print(m)
<_sre.SRE_Match object at 0x01F3FA30>
```

"3 python"이라는 문자열의 첫 번째 문자는 "3"이지만 search는 문자열의 처음부터 검색하는 것이 아니라 문자열 전체를 검색하기 때문에 "3 " 이후의 "python"이라는 문자열과 매치된다.

이렇듯 match 메서드와 search 메서드는 문자열의 처음부터 검색할지의 여부에 따라 다르게 사용해야 한다.

findall

이번에는 findall 메서드를 수행해 보자.

```
>>> result = p.findall("life is too short")
>>> print(result)
['life', 'is', 'too', 'short']
```

"life is too short"라는 문자열의 "life", "is", "too", "short"라는 단어들이 각각 `[a-z]+` 정규식과 매치되어 리스트로 리턴된다.

finditer

이번에는 finditer 메서드를 수행해 보자.

```
>>> result = p.finditer("life is too short")
>>> print(result)
<callable_iterator object at 0x01F5E390>
>>> for r in result: print(r)
...
<_sre.SRE_Match object at 0x01F3F9F8>
<_sre.SRE_Match object at 0x01F3FAD8>
<_sre.SRE_Match object at 0x01F3FAA0>
<_sre.SRE_Match object at 0x01F3F9F8>
```

finditer는 findall과 동일하지만 그 결과로 반복 가능한 객체(iterator object)를 리턴한다. 반복 가능한 객체가 포함하는 각각의 요소는 match 객체이다.

match 객체의 메서드

자, 이젠 match 메서드와 search 메서드를 수행한 결과로 리턴되었던 match 객체에 대해서 알아보자. 앞에서 정규식을 이용한 문자열 검색을 수행하면서 아마도 다음과 같은 궁금증이 생겼을 것이다.

- 어떤 문자열이 매치되었는가?
- 매치된 문자열의 인덱스는 어디서부터 어디까지인가?

match 객체의 메서드들을 이용하면 이 같은 궁금증을 해결할 수 있다. 다음 표를 보자.

method	목적
--------	----

group()	매치된 문자열을 리턴한다.
start()	매치된 문자열의 시작 위치를 리턴한다.
end()	매치된 문자열의 끝 위치를 리턴한다.
span()	매치된 문자열의 (시작, 끝) 에 해당되는 튜플을 리턴한다.

다음의 예로 확인해 보자.

```
>>> m = p.match("python")
>>> m.group()
'python'
>>> m.start()
0
>>> m.end()
6
>>> m.span()
(0, 6)
```

예상한 대로 결과값이 출력되는 것을 확인할 수 있다. match 메서드를 수행한 결과로 리턴된 match 객체의 start()의 결과값은 항상 0일 수밖에 없다. 왜냐하면 match 메서드는 항상 문자열의 시작부터 조사하기 때문이다.

만약 search 메서드를 사용했다면 start()의 값은 다음과 같이 다르게 나올 것이다.

```
>>> m = p.search("3 python")
>>> m.group()
'python'
>>> m.start()
2
>>> m.end()
8
>>> m.span()
(2, 8)
```

[모듈 단위로 수행하기]

지금까지 우리는 re.compile을 이용하여 컴파일된 패턴 객체로 그 이후 작업을 수행했다. re 모듈은 이것을 좀 축약한 형태의 방법을 제공한다. 다음의 예를 보자.

```
>>> p = re.compile('[a-z]+')
>>> m = p.match("python")
```

위 코드가 축약된 형태는 다음과 같다.

```
>>> m = re.match('[a-z]+', "python")
```

위 예처럼 사용하면 컴파일과 match 메서드를 한 번에 수행할 수 있다. 보통 한 번 만든 패턴 객체를 여러 번 사용해야 할 때는 이 방법보다 re.compile을 사용하는 것이 유리하다.

컴파일 옵션

정규식을 컴파일할 때 다음과 같은 옵션을 사용할 수 있다.

- DOTALL = S . 이 줄바꿈 문자를 포함하여 모든 문자와 매치할 수 있도록 한다.
- IGNORECASE = I 대소문자에 관계없이 매치할 수 있도록 한다.
- MULTILINE = M 여러줄과 매치할 수 있도록 한다. (^, \$ 메타문자의 사용과 관계가 있는 옵션이다)
- VERBOSE = X VERBOSE(X) - verbose 모드를 사용할 수 있도록 한다. (정규식을 보기 편하게 만들수 있고 주석등을 사용할 수 있게된다.)

옵션을 사용할 때는 re.DOTALL 처럼 전체 옵션명을 써도 되고 re.S 처럼 약어를 써도 된다.

DOTALL, S

. 메타 문자는 줄바꿈 문자(Wn)를 제외한 모든 문자와 매치되는 규칙이 있다. 만약 Wn 문자도 포함하여 매치하고 싶다면 re.DOTALL 또는 re.S 옵션을 사용해 정규식을 컴파일하면 된다.

다음의 예를 보자.

```
>>> import re
>>> p = re.compile('a.b')
>>> m = p.match('aWnb')
>>> print(m)
None
```

정규식이 a.b인 경우 문자열 aWnb는 매치되지 않음을 알 수 있다. 왜냐하면 Wn은 . 메타 문자와 매치되지 않기 때문이다. Wn 문자와도 매치되게 하려면 다음과 같이 re.DOTALL 옵션을 사용해야 한다.

```
>>> p = re.compile('a.b', re.DOTALL)
>>> m = p.match('aWnb')
>>> print(m)
<_sre.SRE_Match object at 0x01FCF3D8>
```

보통 re.DOTALL은 여러줄로 이루어진 문자열에서 Wn에 상관없이 검색하고자 할 경우에 많이 사용한다.

IGNORECASE, I

re.IGNORECASE 또는 re.I 는 대소문자 구분없이 매치를 수행하고자 할 경우에 사용하는 옵션이다.

다음의 예제를 보자.

```
>>> p = re.compile('[a-z]', re.I)
>>> p.match('python')
<_sre.SRE_Match object at 0x01FCFA30>
>>> p.match('Python')
<_sre.SRE_Match object at 0x01FCFA68>
>>> p.match('PYTHON')
<_sre.SRE_Match object at 0x01FCF9F8>
```

[a-z] 정규식은 소문자만을 의미하지만 re.I 옵션에 의해서 대·소문자 구분 없이 매치된다.

MULTILINE, M

re.MULTILINE 또는 re.M 옵션은 조금 후에 설명할 메타 문자인 `^`, `$`와 연관된 옵션이다. 이 메타 문자에 대해 간단히 설명하자면 `^`는 문자열의 처음을 의미하고, `$`은 문자열의 마지막을 의미한다. 예를 들어 정규식이 `^python`인 경우 문자열의 처음은 항상 python으로 시작해야 매치되고, 만약 정규식이 `python$`이라면 문자열의 마지막은 항상 python으로 끝나야 매치가 된다는 의미이다.

다음 예를 보도록 하자.

```
import re
p = re.compile("^pythonWsWw+")

data = """python one
life is too short
python two
you need python
python three"""

print(p.findall(data))
```

정규식 `^pythonWsWw+` 은 "python"이라는 문자열로 시작하고 그 후에 whitespace, 그 후에 단어가 와야 한다는 의미이다. 검색할 문자열 data는 여러줄로 이루어져 있다.

이 스크립트를 실행하면 다음과 같은 결과가 리턴된다.

```
['python one']
```

`^` 메타 문자에 의해 python이라는 문자열이 사용된 첫 번째 라인만 매치가 된 것이다.

하지만 `^` 메타 문자를 문자열 전체의 처음이 아니라 각 라인의 처음으로 인식시키고 싶은 경우도 있을 것이다. 이럴 때 사용할 수 있는 옵션이 바로 re.MULTILINE 또는 re.M이다. 위 코드를 다음과 같이 수정해 보자.

```
import re
p = re.compile("^pythonWsWw+", re.MULTILINE)

data = """python one
life is too short
python two
you need python
python three"""

print(p.findall(data))
```

re.MULTILINE 옵션으로 인해 `^` 메타 문자가 문자열 전체가 아닌 각 라인의 처음이라는 의미를 갖게 되었다. 이 스크립트를 실행하면 다음과 같은 결과가 출력된다.

```
['python one', 'python two', 'python three']
```

즉, re.MULTILINE 옵션은 `^`, `$` 메타 문자를 문자열의 각 라인마다 적용해 주는 것이다.

VERBOSE, X

지금껏 알아본 정규식은 매우 간단하지만 정규식 전문가들이 만든 정규식을 보면 거의 암호수준이다. 이해하려면 하나하나 조심스럽게 뜯어보아야만 한다. 이렇게 이해하기 어려운 정규식을 주석 또는 라인 단위로 구분할 수 있다면 얼마나 보기 좋고 이해하기 쉬울까? 방법이 있다. 바로 `re.VERBOSE` 또는 `re.X` 옵션을 이용하면 된다.

다음의 예를 보자.

```
charref = re.compile(r'&[#](0[0-7]+|[0-9]+|x[0-9a-fA-F]+);')
```

위 정규식이 쉽게 이해되는가? 이제 다음의 예를 보자.

```
charref = re.compile(r"""
&[#]           # Start of a numeric entity reference
(
    0[0-7]+     # Octal form
  | [0-9]+     # Decimal form
  | x[0-9a-fA-F]+ # Hexadecimal form
)
;             # Trailing semicolon
""", re.VERBOSE)
```

첫 번째와 두 번째 예를 비교해 보면 컴파일된 패턴 객체인 `charref`는 모두 동일한 역할을 한다. 하지만 정규식이 복잡할 경우 두 번째처럼 주석을 적고 여러 줄로 표현하는 것이 훨씬 가독성이 좋다는 것을 알 수 있다.

`re.VERBOSE` 옵션을 사용하면 문자열에 사용된 `whitespace`는 컴파일 시 제거된다(단 `[ ]` 내에 사용된 `whitespace`는 제외). 그리고 줄 단위로 `#`기호를 이용하여 주석문을 작성할 수 있다.

백슬래시 문제

정규표현식을 파이썬에서 사용하려 할 때 혼란을 주게 되는 요소가 한가지 있는데 그것은 바로 백슬래시(`\`)이다.

 패턴

예를들어 LaTeX파일 내에 있는 "`\section`"이라는 문자열을 찾기 위한 정규식을 만든다고 가정해 보자.

다음과 같은 정규식을 생각해 보자.

`\section` 파이썬 인터프리터가 `\s` 먼저 인식. 그 후 `re`가 인식

이 정규식은 `\s` 문자가 `whitespace`로 해석되어 의도한 대로 매치가 이루어지지 않는다.

위 표현은 다음과 동일한 의미가 된다.

```
[ \t\n\r\f\v]ection
```

따라서 위 정규표현식은 다음과 같이 변경되어야 할 것이다.

[`\\section`](#)

즉, 위 정규식에서 사용한 `\` 문자가 문자열 그 자체임을 알려주기 위해 백슬래시 2개를 사용하여 이스케이프 처리를 해야 하는 것이다.

따라서 위 정규식을 컴파일하려면 다음과 같이 작성해야 한다.

```
>>> p = re.compile('WWsection')
```

그런데 여기 또 하나의 문제가 발견된다. 위 처럼 정규식을 만들어서 컴파일 하면 실제 파이썬 정규식 엔진에는 파이썬 문자열 리터럴 규칙에 의하여 `wwO|w` 로 변경되어 `wsection` 이 전달되는 것이다.

※ 이 문제는 위와같은 정규식을 파이썬에서 사용할 때만 적용되는 문제이다. (파이썬의 리터럴 규칙) 유닉스의 grep, vi등에서는 이러한 문제가 없다.

결국 정규식 엔진에 `ww` 문자를 전달하려면 파이썬은 `wwww` 처럼 백슬래시를 4개나 사용해야 한다.

※ 정규식 엔진은 정규식을 해석하고 수행하는 모듈이다.

- w를 문자열로 인식하게 하기
- interpreter에서 w2개를 1개로 인식 -> wwwwww
- re에서도 w2개를 1개로 인식

```
>>> p = re.compile('WWWsection')
```

이렇게 해야만 원하는 결과를 얻을 수 있을 것이다. 하지만 너무 복잡하지 않은가?

만약 위와 같이 **w**를 이용한 표현이 반복되서 사용되는 정규식이라면 너무 복잡하여 이해하기 쉽지 않을 것이다. 이러한 문제로 파이썬 정규식에는 Raw string이라는 것이 생겨나게 되었다. 즉 컴파일 해야 하는 정규식이 Raw String임을 알려줄 수 있도록 파이썬 문법이 만들어진 것이다. 그 방법은 다음과 같다.

>>> p = re.compile(r'WWsection') r은 python interpreter에만 인식해서 re를 위한 w 2개 필요

위와 같이 정규식 문자열 앞에 r문자를 선행하면 이 정규식은 Raw String 규칙에 의하여 백슬래시 2개 대신 1개만 써도 두개를 쓴것과 동일한 의미를 갖게된다.

※ 만약 백슬래시를 사용하지 않는 정규식이라면 r 의 유무에 상관없이 동일한 정규식이 될 것이다.

06-3 강력한 정규 표현식의 세계로

메타문자

아직 살펴보지 않은 메타 문자들에 대해서 모두 살펴보도록 하자. 여기서 다룰 메타 문자들은 앞에서 살펴보았던 메타 문자들과 성격이 조금 다르다. 이전에 살펴본 메타 문자들은 모두 매치되는 문자열들을 **소모시킨다**. 이 소모된다는 말의 의미가 조금 헷갈릴 수 있을 것이다. 문자열이 일단 **소모되어 버리면 그 부분은 검색 대상에서 제외**되지만 소모되지 않는 경우에는 다음에 또 다시 검색 대상이 된다고 생각하면 쉬울 것이다.

`+`, `*`, `[]`, `{}` 등의 메타문자는 매치가 진행될 때 현재 매치되고 있는 문자열의 위치가 변경된다. (보통 소모된다고 표현한다.) 하지만 이와 달리 문자열을 소모시키지 않는 메타 문자들도 있다. 이번에는 이런 문자열 소모가 없는(zero-width assertions) 메타 문자들에 대해서 살펴보기로 하자.

|

| 메타문자는 **"or"**의 의미와 동일하다. `A|B` 라는 정규식이 있다면 이것은 A 또는 B라는 의미가 된다.

```
>>> p = re.compile('Crow|Servo')
>>> m = p.match('CrowHello')
>>> print(m)
<_sre.SRE_Match object: span=(0, 4), match='Crow'>
```

^ 시작하는

^ 메타문자는 문자열의 맨 처음과 일치함을 의미한다. 이전에 알아보았던 컴파일 옵션 `re.MULTILINE` 을 사용할 경우에는 여러줄의 문자열에서는 각 라인의 처음과 일치하게 된다.

다음의 예를 보자.

```
                "life"로 시작?
>>> print(re.search('^Life', 'Life is too short'))
<_sre.SRE_Match object at 0x01FCF3D8>
>>> print(re.search('^Life', 'My Life'))
None
```

`^Life` 정규식은 "Life"라는 문자열이 처음에 온 경우에는 매치하지만 처음 위치가 아닌 경우에는 매치되지 않음을 알 수 있다.

\$ 끝나는

\$ 메타문자는 ^ 메타문자의 반대의 경우이다. \$는 문자열의 끝과 매치함을 의미한다.

다음의 예를 보자.

```
                short로 끝나니?
>>> print(re.search('short$', 'Life is too short'))
<_sre.SRE_Match object at 0x01F6F3D8>
>>> print(re.search('short$', 'Life is too short, you need python'))
```

None

`short$` 정규식은 검색할 문자열의 "short"로 끝난 경우에는 매치되지만 그 이외의 경우에는 매치되지 않음을 알 수 있다.

※ `^` 또는 `$` 문자를 메타문자가 아닌 문자 그 자체로 매치하고 싶은 경우에는 `[^]`, `[$]` 처럼 사용하거

나 `w^`, `w$` 로 사용하면 된다.

`[^]`: `^`에 다른 문자까지 있으면 `not` 의미

`'^'`: 일반 문자(기능 탈출(escape `w`))

WA

`WA`는 문자열의 처음과 매치됨을 의미한다. `^`와 동일한 의미이지만 `re.MULTILINE` 옵션을 사용할 경우에는 다르게 해석된다. `re.MULTILINE` 옵션을 사용할 경우 `^`은 라인별 문자열의 처음과 매치되지만 `WA`는 라인과 상관없이 전체 문자열의 처음하고만 매치된다.

WZ

`WZ`는 문자열의 끝과 매치됨을 의미한다. 이것 역시 `WA`와 동일하게 `re.MULTILINE` 옵션을 사용할 경우 `$` 메타문자와는 달리 전체 문자열의 끝과 매치된다.

Wb

`Wb`는 단어 구분자(Word boundary)이다. 보통 단어는 whitespace에 의해 구분이 된다. 다음의 예를 보자.

경계 + " " 경계 + " "

```
>>> p = re.compile(r'WbclassWb')      특정 단어 찾을 때 효과적
>>> print(p.search('no class at all'))
<_sre.SRE_Match object at 0x01F6F3D8>
```

`WbclassWb` 정규식은 "class"라는 단어와 매치됨을 의미한다. 따라서 `no class at all`의 `class`라는 단어와 매치됨을 확인할 수 있다.

```
>>> print(p.search('the declassified algorithm'))
None      앞 뒤 공백이 없어서 찾을 수 x
```

위 예의 `the declassified algorithm`라는 문자열 안에 `class`라는 문자열이 포함되어 있긴 하지만 whitespace로 구분된 단어가 아니므로 매치되지 않는다.

```
>>> print(p.search('one subclass is'))
None
```

`subclass`라는 문자열 역시 `class`앞에 `sub`라는 문자열이 더해져 있으므로 매치되지 않음을 알 수 있다.

`Wb` 메타문자를 이용할 경우 주의해야 할 점이 한가지 있다. `Wb`는 파이썬 리터럴 규칙에 의하면 백스페이스(Back Space)를 의미하므로 백스페이스가 아닌 Word Boundary임을 알려주기 위해 `r'WbclassWb'` 처럼 raw string임을 알려주는 기호 `r`을 반드시 붙여주어야 한다.

WB

WB 메타문자는 **Wb** 메타문자의 반대의 경우이다. 즉, whitespace로 구분된 단어가 아닌 경우에만 매치된다.

```
>>> p = re.compile(r'WBclassWB')
>>> print(p.search('no class at all'))
None
>>> print(p.search('the declassified algorithm'))
<_sre.SRE_Match object at 0x01F6FA30>
>>> print(p.search('one subclass is'))
None
```

class라는 문자열 좌우에 whitespace가 있는 경우에는 매치가 안되는 것을 확인할 수 있다.

그룹핑

ABC라는 문자열이 계속해서 반복되는지 조사하는 정규식을 작성하고 싶다고 하자. 어떻게 해야 할까? 지금까지 공부한 내용으로는 위 정규식을 작성할 수 없다. 이럴 때 필요한 것이 바로 그룹핑(Grouping)이다.

위의 경우는 다음처럼 그룹핑을 이용하여 작성할 수 있다.

(ABC)+

그룹을 만들어 주는 메타문자는 바로 (과)이다.

```
>>> p = re.compile('(ABC)+')
>>> m = p.search('ABCABCABC OK?')
>>> print(m)
<_sre.SRE_Match object at 0x01F7B320>
>>> print(m.group())
ABCABCABC
```

다음의 예를 보자.

Ww: 알파벳, 숫자 +:1자 이상
Ws: 공백 +:1개 이상
Wd: 숫자 +:1개 이상

```
>>> p = re.compile(r"Ww+Ws+Wd+[-]Wd+[-]Wd+")
>>> m = p.search("park 010-1234-1234")
```

Ww+Ws+Wd+[-]Wd+[-]Wd+은 이름 + " " + 전화번호 형태의 문자열을 찾는 정규표현식이다. 그런데 이렇게 매치된 문자열 중에서 **이름만 뽑아내고 싶다면** 어떻게 해야 할까?

보통 반복되는 문자열을 찾을 때 그룹을 이용하는데, 그룹을 이용하는 보다 큰 이유는 위에서 볼 수 있듯이 매치된 문자열 중에서 특정 부분의 문자열만 뽑아내기 위해서인 경우가 더 많다.

위 예에서 만약 "이름" 부분만을 뽑아내려 한다면 다음과 같이 할 수 있다.

```
>>> p = re.compile(r"(Ww+)Ws+Wd+[-]Wd+[-]Wd+")
>>> m = p.search("park 010-1234-1234")
>>> print(m.group(1))
park
```

이름에 해당되는 Ww+ 부분을 그룹((Ww+))으로 만들면 match object의 group(index) 메서드를 이용하여 그룹핑된 부분의 문자열만 뽑아낼 수 있다. group 메소드의 index는 다음과 같은 의미를 갖는다.

group(인덱스)	설명
group(0)	매치된 전체 문자열
group(1)	첫 번째 그룹에 해당되는 문자열
group(2)	두 번째 그룹에 해당되는 문자열
group(n)	n 번째 그룹에 해당되는 문자열

다음의 예제를 계속해서 보자.

```
>>> p = re.compile(r"(Ww+)Ws+(Wd+[-]Wd+[-]Wd+)")
>>> m = p.search("park 010-1234-1234")
>>> print(m.group(2))
010-1234-1234
```

이번에는 전화번호 부분을 추가로 그룹((Wd+[-]Wd+[-]Wd+))으로 만들었다. 이렇게 하면 group(2)처럼 사용하여 전화번호만을 뽑아낼 수 있다.

만약 전화번호 중에서 국번만 뽑아내고 싶으면 어떻게 해야 할까? 다음과 같이 국번 부분을 또 그룹핑하면 된다.

```
>>> p = re.compile(r"(Ww+)Ws+((Wd+[-]Wd+[-]Wd+))")
>>> m = p.search("park 010-1234-1234")
>>> print(m.group(3))
010
```

위 예에서 보듯이 (Ww+)Ws+((Wd+[-]Wd+[-]Wd+)) 처럼 그룹을 중첩되게 사용하는 것도 가능하다. 그룹이 중첩되어 사용되는 경우는 바깥쪽부터 시작하여 안쪽으로 들어갈수록 인덱스가 증가한다.

그룹핑된 문자열 재참조하기

그룹의 또 하나 좋은 점은 한번 그룹핑된 문자열을 재참조(Backreferences)할 수 있다는 점이다. 다음의 예를 보자.

```
>>> p = re.compile(r'(WbWw+)Ws+W1')
>>> p.search('Paris in the the spring').group()
'the the'
```

정규식 (WbWw+)Ws+W1은 (그룹1) + " " + "그룹1과 동일한 단어" 와 매치됨을 의미한다. 이렇게 정규식을 만들게 되면 2개의 동일한 단어가 연속적으로 사용되어야만 매치되게 된다. 이것을 가능하게 해 주는 것이 바로 재 참조 메타문자인 W1이다. W1은 정규식의 그룹중 첫번째 그룹을 지칭한다.

※ 두번째 그룹을 참조하려면 W2를 사용하면 된다.

그룹핑된 문자열에 이름 붙이기

정규식 내에 그룹이 무척 많아진다고 가정해 보자. 예를 들어 정규식 내에 그룹이 10개 이상만 되어도 매우 혼란스러울 것이다. 거기에 더해 정규식이 수정되면서 그룹이 추가, 삭제되면 그 그룹을 인덱스

로 참조했던 프로그램들도 모두 변경해 주어야 하는 위험도 갖게 된다. 만약 그룹을 인덱스가 아닌 이름(Named Groups)으로 참조할 수 있다면 어떨까? 그렇다면 이런 문제들에서 해방되지 않을까?

이러한 이유로 정규식은 그룹을 만들 때 그룹명을 지정할 수 있게 했다. 그 방법은 다음과 같다.

```
(?P<name>Ww+)Ws+((Wd+)[-]Wd+)[-]Wd+
```

위 정규식은 이전에 보았던 이름과 전화번호를 추출하는 정규식이다. 기존과 달라진 부분은 다음과 같다.

```
(Ww+) --> (?P<name>Ww+)
```

대단히 복잡해 진 것처럼 보이지만 (Ww+) 라는 그룹에 "name"이라는 이름을 붙인 것에 불과하다. 여기서 사용한 (?...) 표현식은 정규표현식의 확장구문이다. (여기서 ...은 다양하게 변한다는 anything의 의미이다.) 이 확장구문을 이용하기 시작하면 가독성이 무척 떨어지긴 하지만 반면에 강력함을 갖게 된다.

그룹에 이름을 지어주기 위해서는 다음과 같은 확장구문을 사용해야 한다.

```
(?P<그룹명>...)
```

그룹에 이름을 지정하고 참조하는 다음의 예를 보자.

```
>>> p = re.compile(r"(?P<name>Ww+)Ws+((Wd+)[-]Wd+)[-]Wd+")
>>> m = p.search("park 010-1234-1234")
>>> print(m.group("name"))
park
```

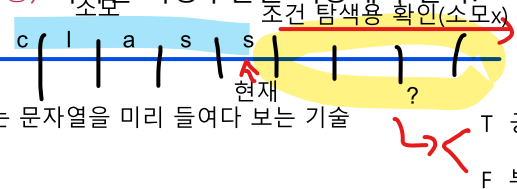
위 예에서 볼 수 있듯이 name이라는 그룹명으로 참조할 수 있다.

그룹명을 이용하면 정규식 내에서 재참조하는 것도 가능하다.

```
>>> p = re.compile(r'(?P<word>WbWw+)Ws+(?P=word)')
>>> p.search('Paris in the the spring').group()
'the the'
```

위 예에서 보듯이 재 참조시에는 (?P=그룹명)이라는 확장구문을 이용해야 한다.

전방 탐색 현재 위치에서 앞에 있는 문자열을 미리 들여다 보는 기술



정규식에 막 입문한 사람들이 가장 어려워하는 것이 바로 전방 탐색(Lookahead Assertions) 확장 구문이다. 정규식 안에 이 확장 구문이 사용되면 순식간에 암호문처럼 알아보기 어렵게 바뀌기 때문이다. 하지만 이 전방 탐색이 꼭 필요한 경우가 있으며 매우 유용한 경우도 많으니 꼭 알아 두도록 하자.

다음의 예제를 보자.

```
(1개 이상 모든 문자) :
>>> p = re.compile("."+":")
>>> m = p.search("http://google.com")
>>> print(m.group())
http:
```

정규식 `.+:` 과 일치하는 문자열로 "http:"가 리턴되었다. 하지만 "http:" 라는 검색 결과에서 ":"을 제외하고 출력하려면 어떻게 해야 할까? 위 예는 그나마 간단하지만 훨씬 복잡한 정규식이어서 그룹핑은 추가로 할 수 없다는 조건까지 더해진다면 어떻게 해야 할까?

이럴 때 사용할 수 있는 것이 바로 전방 탐색이다. 전방 탐색에는 긍정(Positive)과 부정(Negative)의 2 종류가 있고 다음과 같이 표현된다.

- 긍정형 전방 탐색(`(?=...)`) - ... 에 해당되는 정규식과 매치되어야 하며 조건이 통과되어도 문자열이 소모되지 않는다.
! : not
- 부정형 전방 탐색(`(?!...)`) - ...에 해당되는 정규식과 매치되지 않아야 하며 조건이 통과되어도 문자열이 소모되지 않는다.

긍정형 전방 탐색

긍정형 전방 탐색을 이용하면 http:의 결과를 http로 바꿀 수 있다. 다음의 예를 보자.

```
.,+만 함. 그 후에 앞에 :이 있는지 보라 (:은 기준일뿐)
>>> p = re.compile(".+(?=:)") 한 글자씩 확인 (h기준 t확인. : 아니니 t로 가서 다음 t 확인 ....p로 가서 다음 : 확인. 있음.
>>> m = p.search("http://google.com") 여기서
>>> print(m.group())
http
```

정규식 중 `:`에 해당하는 부분이 긍정형 전방탐색 기법이 적용되어 `(?=:)` 으로 변경되었다. 이렇게 되면 기존 정규식과 검색에서는 동일한 효과를 발휘하지만 `:`에 해당되는 문자열이 정규식 엔진에 의해 소모되지 않아(검색에는 포함되지만 검색 결과에는 제외됨) 검색 결과에서는 `:`이 제거된 후 리턴되는 효과가 있다.

자, 이번에는 다음의 정규식을 보자.

```
.*[.].*$
```

이 정규식은 `파일명 + '.' + 확장자` 를 나타내는 정규식이다. 이 정규식은 `foo.bar`, `autoexec.bat`, `sendmail.cf`등과 매치할 것이다.

자, 이제 이 정규식에 확장자가 "bat"인 파일은 제외해야 한다"는 조건을 추가해 보자. 가장 먼저 생각할 수 있는 정규식은 다음과 같을 것이다.

```
.*[.](^b).*$
```

이 정규식은 확장자가 b라는 문자로 시작하면 안된다는 의미이다. 하지만 이 정규식은 `foo.bar`라는 파일마저 걸러내 버린다. 다시 정규식을 다음과 같이 수정 해 보자.

```
.*[.](^b|..|^a|..|^t)$
```

이 정규식은 | 메타 문자를 사용하여 확장자의 첫 번째 문자가 b가 아니거나 두 번째 문자가 a가 아니거나 세 번째 문자가 t가 아닌 경우를 의미한다. 이 정규식에 의하여 `foo.bar`는 제외되지 않고 `autoexec.bat`은 제외되어 만족스러운 결과를 리턴한다. 하지만 이 정규식은 아쉽게도 `sendmail.cf`처럼 확장자의 문자 개수가 2개인 케이스를 포함하지 못 하는 오동작을 하기시작한다.

자, 이제 다음과 같이 바꾸어야 한다.

```
.*[.](^[^b].?|.^[^a].?|..^[^t]?)$
```

확장자의 문자 개수가 2개여도 통과되는 정규식이 만들어졌다. 하지만 정규식은 점점 더 복잡해지고 이해하기 어려워진다.

자, 그런데 여기서 bat 파일 말고 exe 파일도 제외하라는 조건이 추가로 생긴다면 어떻게 될까? 이 모든 조건을 만족하는 정규식을 구현하려면 패턴은 더욱더 복잡해져야만 할 것이다.

부정형 전방 탐색

이러한 상황의 구원투수는 바로 "부정형 전방탐색"이다. 위 케이스는 부정형 전방탐색을 사용하면 다음과 같이 간단하게 처리된다.

```
.*[.](?!bat$).*$(모든문자 여러 개). $(끝나는)  
bat로 끝나는 확장자는 빼라
```

확장자가 bat가 아닌 경우에만 통과된다는 의미이다. bat라는 문자열이 있는지 조사하는 과정에서 문자열이 소모되지 않으므로 bat가 아니라고 판단되면 그 이후 정규식 매칭이 진행된다.

exe 역시 제외하라는 조건이 추가되더라도 다음과 같이 간단히 표현할 수 있다.

```
.*[.](?!bat$|exe$).*$(모든문자 여러 개). $(끝나는)
```

문자열 바꾸기

sub 메서드를 이용하면 정규식과 매치되는 부분을 다른 문자로 쉽게 바꿀 수 있다.

다음의 예를 보자.

```
>>> p = re.compile('(blue|white|red)')  
>>> p.sub('colour', 'blue socks and red shoes')  
'colour socks and colour shoes'
```

sub 메서드의 첫 번째 입력 인수는 "바꿀 문자열(replacement)"이 되고, 두 번째 입력 인수는 "대상 문자열"이 된다. 위 예에서 볼 수 있듯이 blue 또는 white 또는 red라는 문자열이 colour라는 문자열로 바뀌는 것을 확인할 수 있다.

그런데 딱 한 번만 바꾸고 싶은 경우도 있다. 이렇게 바꾸기 횟수를 제어하려면 다음과 같이 세 번째 입력 인수로 count 값을 넘기면 된다.

```
>>> p.sub('colour', 'blue socks and red shoes', count=1)  
'colour socks and red shoes'
```

처음 일치하는 blue만 colour라는 문자열로 한 번만 바꾸기가 실행됨을 알 수 있다.

[sub 메서드와 유사한 subn 메서드]

subn 역시 sub와 동일한 기능을 하지만 리턴되는 결과를 튜플로 리턴한다는 차이가 있다. 리턴된 튜플의 첫 번째 요소는 변경된 문자열이고, 두 번째 요소는 바꾸기가 발생한 횟수이다.

```
>>> p = re.compile('(blue|white|red)')
>>> p.subn('colour', 'blue socks and red shoes')
('colour socks and colour shoes', 2)
```

sub 메서드 사용 시 참조 구문 사용하기

sub 메서드를 사용할 때 참조 구문을 사용할 수 있다. 다음의 예를 보자.

```
>>> p = re.compile(r"(?P<name>Ww+)Ws+(?P<phone>(Wd+)[-]Wd+[-]Wd+)")
>>> print(p.sub("Wg<phone> Wg<name>", "park 010-1234-1234"))
010-1234-1234 park
```

위 예는 이름 + 전화번호의 문자열을 전화번호 + 이름으로 바꾸는 예이다. sub의 바꿀 문자열 부분에 Wg<그룹명>을 이용하면 정규식의 그룹명을 참조할 수 있게된다.

다음과 같이 그룹명 대신 참조번호를 이용해도 마찬가지로 결과가 리턴된다.

```
>>> p = re.compile(r"(?P<name>Ww+)Ws+(?P<phone>(Wd+)[-]Wd+[-]Wd+)")
>>> print(p.sub("Wg<2> Wg<1>", "park 010-1234-1234"))
010-1234-1234 park
```

sub 메서드의 입력 인수로 함수 넣기

sub 메서드의 첫 번째 입력 인수로 함수를 넣을 수도 있다. 다음의 예를 보자.

```
>>> def hexrepl(match):
...     "Return the hex string for a decimal number"
...     value = int(match.group())
...     return hex(value)
...
>>> p = re.compile(r'Wd+')
>>> p.sub(hexrepl, 'Call 65490 for printing, 49152 for user code.')
'Call 0xffd2 for printing, 0xc000 for user code.'
```

hexrepl 함수는 match 객체(위에서 숫자에 매치되는)를 입력으로 받아 16진수로 변환하여 리턴하는 함수이다. sub의 첫 번째 입력 인수로 함수를 사용할 경우 해당 함수의 첫 번째 입력 인수에는 정규식과 매치된 match 객체가 입력된다. 그리고 매치되는 문자열은 함수의 리턴값으로 바뀌게 된다.

Greedy vs Non-Greedy

정규식에서 Greedy(탐욕스러운)란 어떤 의미일까? 다음의 예제를 보자.

```
>>> s = '<html><head><title>Title</title>'
>>> len(s)
32
>>> print(re.match('<.*>', s).span())
(0, 32)
>>> print(re.match('<.*>', s).group())
<html><head><title>Title</title>
```

<.*> 정규식의 매치결과로 <html> 문자열이 리턴되기를 기대했을 것이다. 하지만 * 메타문자는 매우

탐욕스러워서 매치할 수 있는 최대한의 문자열인 `<html><head><title>Title</title>` 문자열을 모두 소모시켜 버렸다. 어떻게 하면 이 탐욕스러움을 제한하고 `<html>` 이라는 문자열까지만 소모되도록 막을 수 있을까?

다음과 같이 non-greedy 문자인 `?`을 사용하면 `*`의 탐욕을 제한할 수 있다.

```
>>> print(re.match('<.*?>', s).group())
<html>
```

non-greedy 문자인 `?`은 `*?`, `+?`, `??`, `{m,n}?`과 같이 사용할 수 있다. 가능한 한 가장 최소한의 반복을 수행하도록 도와주는 역할을 한다.

06-4 파이썬으로 XML 처리하기

이 절에서는 파이썬으로 XML 문서를 다루는 방법에 대해서 살펴본다. XML 처리를 위한 파이썬 라이브러리는 아주 많으며 아래 사이트에서 확인할 수 있다.

<http://wiki.python.org/moin/PythonXml>

※ 이 절은 XML에 대한 기초적인 지식이 있는 사람을 대상으로 한다. XML 처리에 대해 굳이 알아야 할 필요가 없다면 건너뛰어도 좋다.

이 책에서는 가장 많은 이들의 사랑을 받고 있는 라이브러리인 ElementTree를 사용하는 방법에 대해서 다룬다. ElementTree는 Tkinter로 유명한 프레드릭 런드(Fredrik Lundh)가 만든 XML 제너레이터 & 파서이다.

※ ElementTree는 외부 라이브러리로 존재하다가 파이썬 2.5부터 통합되었다.

XML 문서 생성하기

ElementTree를 이용하여 다음과 같은 구조의 XML문서를 생성해 보자.

```
<note date="20120104">
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
</note>
```

먼저 다음과 같은 소스를 작성해 보자:

```
from xml.etree.ElementTree import Element, dump

note = Element("note")
to = Element("to")
to.text = "Tove"

note.append(to)
dump(note)
```

위 소스의 실행 결과는 다음과 같다:

```
<note><to>Tove</to></note>
```

엘리먼트(Element)를 이용하면 태그를 만들 수 있고, 만들어진 태그에 텍스트 값을 추가할 수 있음을 알 수 있다.

SubElement

서브엘리먼트(SubElement)를 이용하면 조금 더 편리하게 태그를 추가할 수 있다.

```
from xml.etree.ElementTree import Element, SubElement, dump

note = Element("note")
to = Element("to")
to.text = "Tove"

note.append(to)
SubElement(note, "from").text = "Jani"

dump(note)
```

실행 결과는 다음과 같다.

```
<note><to>Tove</to><from>Jani</from></note>
```

서브엘리먼트는 태그명과 태그의 텍스트 값을 한 번에 설정할 수 있다.

또는 다음과 같이 태그 사이에 태그를 추가하거나 특정 태그를 삭제할 수도 있다.

```
dummy = Element("dummy")
note.insert(1, dummy)
note.remove(dummy)
```

위의 예는 dummy라는 태그를 삽입하고 삭제하는 경우이다.

애트리뷰트 추가하기

이번에는 note 태그에 애트리뷰트(attribute)를 추가해 보자.

```
from xml.etree.ElementTree import Element, SubElement, dump

note = Element("note")
to = Element("to")
to.text = "Tove"

note.append(to)
SubElement(note, "from").text = "Jani"
note.attrib["date"] = "20120104"

dump(note)
```

note.attrib["date"]와 같은 방법으로 애트리뷰트 값(attribute value)을 추가할 수 있다.

다음과 같이 엘리먼트 생성 시 직접 애트리뷰트 값을 추가하는 방법을 사용해도 된다.

```
note = Element("note", date="20120104")
```

실행 결과는 다음과 같다.

```
<note date="20120104"><to>Tove</to><from>Jani</from></note>
```

이상으로 XML 태그와 애트리뷰트를 추가하는 방법에 대해서 알아보았다. 완성된 소스는 다음과 같다.

```
from xml.etree.ElementTree import Element, SubElement, dump
```

```

note = Element("note")
note.attrib["date"] = "20120104"

to = Element("to")
to.text = "Tove"
note.append(to)

SubElement(note, "from").text = "Jani"
SubElement(note, "heading").text = "Reminder"
SubElement(note, "body").text = "Don't forget me this weekend!"
dump(note)

```

실행 결과는 다음과 같다.

```
<note date="20120104"><to>Tove</to><from>Jani</from>...생략...</note>
```

indent 함수

위 결과값은 한 줄로 이어져 있어 보기가 쉽지 않다. 정렬된 형태의 xml 값을 보려면 다음과 같이 indent 함수를 이용해 보자.

```

from xml.etree.ElementTree import Element, SubElement, dump

note = Element("note")
note.attrib["date"] = "20120104"

to = Element("to")
to.text = "Tove"
note.append(to)

SubElement(note, "from").text = "Jani"
SubElement(note, "heading").text = "Reminder"
SubElement(note, "body").text = "Don't forget me this weekend!"

def indent(elem, level=0):
    i = "\n" + level*" "
    if len(elem):
        if not elem.text or not elem.text.strip():
            elem.text = i + " "
        if not elem.tail or not elem.tail.strip():
            elem.tail = i
        for elem in elem:
            indent(elem, level+1)
        if not elem.tail or not elem.tail.strip():
            elem.tail = i
    else:
        if level and (not elem.tail or not elem.tail.strip()):
            elem.tail = i

indent(note)
dump(note)

```

indent 함수를 이용하면 다음과 같이 정렬된 형태의 xml 값을 확인할 수 있다.

```

<note date="20120104">
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>

```



```
<body>Don't forget me this weekend!</body>
</note>
```

파일에 쓰기(write) 수행하기

이제 생성한 xml 문서를 가지고 다음과 같이 엘리먼트의 write 메서드를 이용하여 파일에 쓰기를 해보자.

```
from xml.etree.ElementTree import ElementTree
ElementTree(note).write("note.xml")
```

note.xml이 생성되는 것을 확인할 수 있다.

XML문서 파싱하기

이번에는 생성된 XML 문서를 파싱(parsing)하고 검색하는 방법에 대해서 알아보자.

```
from xml.etree.ElementTree import parse
tree = parse("note.xml")
note = tree.getroot()
```

ElementTree의 parse라는 함수를 이용하여 xml을 파싱할 수 있다.

애트리뷰트 값 읽기

어트리뷰트 값은 다음과 같이 읽을 수 있다:

```
print(note.get("date"))
print(note.get("foo", "default"))
print(note.keys())
print(note.items())
```

get 메서드는 애트리뷰트의 값을 읽는다. 만약 두 번째 입력 인수로 디폴트 값을 주었다면 첫 번째 인자에 해당되는 애트리뷰트 값이 없을 경우 두 번째 값을 리턴한다.

keys는 모든 애트리뷰트의 키 값을 리스트로 리턴하고, items는 key, value 쌍을 리턴한다. 애트리뷰트 값을 가져오는 방법은 앞에서 배운 딕셔너리와 동일함을 알 수 있다.

결과는 다음과 같다.

```
20120104
default
['date']
[('date', '20120104')]
```

XML 태그 접근하기

XML 태그에 접근하는 방법은 다음과 같다

```
from_tag = note.find("from")
from_tags = note.findall("from")
```

```
from_text = note.findtext("from")
```

`note.find("from")`은 `note` 태그 하위에 `from`과 일치하는 첫 번째 태그를 찾아서 리턴하고, 없으면 `None`을 리턴한다. `note.findall("from")`은 `note` 태그 하위에 `from`과 일치하는 모든 태그를 리스트로 리턴한다. `note.findtext("from")`은 `note` 태그 하위에 `from`과 일치하는 첫 번째 태그의 텍스트 값을 리턴한다.

특정 태그의 모든 하위 엘리먼트를 순차적으로 처리할 때는 아래의 메서드를 사용한다.

```
childs = note.getiterator()  
childs = note.getchildren()
```

`getiterator()` 함수는 첫 번째 인수로 다음과 같이 태그명을 받을 수도 있다.

```
note.getiterator("from")
```

위와 같은 경우 `from` 태그의 하위 엘리먼트들이 순차적으로 리턴되며, 보통 다음과 같이 많이 사용된다.

```
for parent in tree.getiterator():  
    for child in parent:  
        ... work on parent/child tuple
```

이상으로 파이썬에서 XML을 처리하기 위한 `ElementTree`의 사용 방법에 대해서 간략히 알아보았다. 더 자세한 내용은 프레드릭 런드가 직접 작성한 튜토리얼에서 확인하기 바란다.

<http://effbot.org/zone/element.htm>